DeLonghi PrimaDonna ELITE ��� BLE MITM (������������ ������������)

DeLonghi PrimaDonna ELITE — BLE MITM міст + Home Assistant + WLED-матриця + радар присутності

Останнє оновлення: вересень 2026, прошивка **v4.58-C6-MITM-gaps**

🔄 Як користуватись цим документом у новому чаті

Встав весь цей файл на початку нового чату разом із фразою: “Продовжуємо проєкт DeLonghi BLE MITM, ось повна документація поточного стану”. Claude одразу зрозуміє весь контекст: протокол, що реалізовано, як компілювати, які автоматизації потрібні.

Якщо потрібен сам код прошивки — прикріпи файл `delonghi_c6_mitm_v458_gaps.ino` (чи новішу версію, якщо вже змінювали) до повідомлення.

1️⃣ Опис роботи приладу

Що це

ESP32-C6 виступає як MITM-міст (man-in-the-middle) між: - Кавомашиною DeLonghi PrimaDonna ELITE ECAM 656.75 (MAC `00:1E:C0:30:C0:45`, серійник `219006CP16031740064`) - Офіційним застосунком CoffeeLink на телефоні

ESP32-C6 одночасно: - **central-роль** → підключається до реальної машини по BLE - **peripheral-роль** → рекламує себе телефону як клоновану машину (той самий MAC, той самий сервіс)

Завдяки цьому міст бачить усі команди й відповіді між застосунком і машиною, і водночас може сам ін’єктувати власні команди (наприклад, від Home Assistant), не заважаючи офіційному застосунку працювати паралельно.

Той самий ESP32-C6 додатково опитує окремий, фізично підключений **радар присутності LD2410C** (другий, незалежний UART) — детально в розділі 8.

Протокол кавомашини (повністю розшифрований)

Кожна команда починається з `0d` (старт), далі довжина, опкод, дані, і чек-сума = XOR усіх попередніх байтів XOR `0x55`

Родина опкодів 82 f0 — приготування напою:

```
0d 0b 82 f0 <recipeID> <action> <вода_старший> <вода_молодший> <молоко_старший> <молоко_молодший>
<міцність> <checksum>
```

`action`: `0x00`=зберегти, `0x01`=старт, `0x04`=стоп Вода/кава: мл×2 (16-біт). Молоко: сек×10 (16-біт). Міцність: `(рівень×16) XOR mode`, де `mode`=1 для SAVE, =2 для СТАРТ/СТОП `recipeId`: еспресо=1, кава=2, капучино=7, латте макіато=8, гаряча вода=16

A9 F0 — перемкнути активний профіль на машині: `0d 05 A9 F0 <номер_профілю_1-6> <checksum>`

A6 F0 — прочитати поточні параметри напою з активного профілю: `0d 07 A6 F0 <номер_профілю> <recipeId> <recipeId> <checksum>` → відповідь 11 байт: `d0 0a A6 F0 <профіль> <вода_ст> <вода_мол> <молоко_ст> <молоко_мол> <міцність_комбо> <checksum>`

0x75 (26 байт) — основний статус-пакет, підтверджений живими тестами: - `byte[1]` = заявлена довжина всього пакета (`0x1a`=26) — використовується для точного визначення межі BLE-фрагментів (див. розділ 7) - `byte[9]` (маскований `&0x07`) = базовий стан: `0x00`/`0x01`=сон, `0x03`=прогрів, `0x05`=готова - `byte[19]` (`func`) / `byte[20]` (`exec`) — внутрішній код активності машини: - `func==0` → авторитетний сигнал справжнього сну (підтверджено декомпіляцією офіційного CoffeeLink) - `func==7, exec==0` → “готова” (і водночас “щойно завершила” — розрізняється лише за %, не за самим func/exec) - `func==7, exec≠0` → активна видача кави/еспресо/капучино/латте (родина, що йде через опкод 82F0 з recipeId 1/2/7/8) - `func==11, exec==3` → **підтверджено живими тестами** — видача гарячої води (окремий код, не через func==7!) - `func==1,2,5,10,16,17` та інші — інші проміжні/недокументовані фази (прогрів, помол, перехідні стани) — не всі остаточно картовані - `byte[25]` = % приготування напою (0-100)

SAVE (`action=0x00`) реально запишеться в постійну пам’ять машини **лише** якщо `func==7 && exec==0` в останньому статус-пакеті (те саме, що офіційний застосунок перевіряє перед збереженням).

Прошивка вміє (детальніше в розділі 3)

- Показувати кавові напої, варити з обраними параметрами
- Зберігати “свої” параметри локально (в ESP32) БЕЗ впливу на машину
- Записувати параметри напряму в один із 6 профілів машини
- Читати поточні параметри з будь-якого профілю (з візуальним порівнянням “було/стало”)
- Обирати джерело параметрів для приготування “на льоту”: власна пам’ять ESP32, чи один із 6 профілів машини
- Детектувати варіння (включно з гарячою водою) і публікувати % наповнення в реальному часі
- Опитувати радар присутності LD2410C, публікувати два незалежні стани (рух/присутність) у MQTT

---

2 Апаратна частина

ESP32-C6-WROOM-1 — основні піни

Призначення	Пін
RGB-статус-індикатор (WS2812/NeoPixel)	GPIO8
Радар LD2410C — RX (приймає від датчика)	GPIO6
Радар LD2410C — TX (шле команди датчику)	GPIO7

⚠ Ці піни підібрані як типово вільні для звичайної плати ESP32-C6 dev board, але **не перевірені на конкретно твоїй платі** — звір з підписами на своїй платі перед підключенням нового обладнання (уникай strapping-пінів, зазвичай 4/5/8/9/15).

Підключення радара LD2410C

Модуль позначений **“HLK-LD2410C V1.1”**, основний ряд пінів: `TX RX OUT GND VCC`

LD2410C	ESP32-C6
VCC	5V
GND	GND
TX (датчика)	**GPIO6** (RX ESP32)
RX (датчика)	**GPIO7** (TX ESP32)
OUT	не використовується (простий HIGH/LOW-вихід, ми читаємо через UART)

На зворотному боці плати є ще одна пара `VIN/GND` — це службовий/тестовий заводський роз'єм, не потрібен.

Розміщення: 24 ГГц радар не конфліктує з 2.4 ГГц Wi-Fi/BLE ESP32. Можна ставити впритул до плати ESP32, головне — не заступати антени металом (пластиковий корпус перед антенами не заважає, радіохвилі проходять крізь нього вільно).

Додатковий, не задіяний нами компонент: на платі є CdS-фоторезистор (світлодатчик), використовується внутрішньою логікою сенсора для керування пином OUT залежно від освітлення. Значення доступне лише в "інженерному режимі" протоколу — наша бібліотека його не читає, не інтегровано.

3️⃣ Компіляція — інструменти та бібліотеки

Середовище

- **arduino-cli 1.1.1**
- **ESP32 core 3.2.1** (`esp32:esp32`)
- Плата: `esp32:esp32:esp32c6`

Прапорці компіляції (важливо — саме `CDCOnBoot=default`, не `cdc`!)

```
        arduino-cli compile --fqbn "esp32:esp32:esp32c6:PartitionScheme=min_spiffs,CDCOnBoot=default"
<папка_проєкту> --export-binaries
```

⚠ **`CDCOnBoot=default` (вимкнено), а не `CDCOnBoot=cdc`** — з увімкненим CDC і без підключеного USB-хоста внутрішній буфер `Serial.print()` заповнюється й усі подальші виклики зависають назавжди (підтверджено живим тестом — блокувало весь `loop()`, і разом з ним варіння кави). Якщо колись знадобиться жива USB-діагностика — візьми будь-яку версію **до v4.42** (усі мали `CDCOnBoot=cdc`), а поточну стабільну після цього поверни назад.

Бібліотеки (усі через `git clone` в `~/Arduino/libraries/`)

Бібліотека	Джерело	Версія/гілка
NimBLE-Arduino	`h2zero/NimBLE-Arduino`	гілка **2.3.4** (не master!)
PubSubClient	`knolleary/pubsubclient`	останній
WiFiManager	`tzapu/WiFiManager`	останній
Adafruit_NeoPixel	`adafruit/Adafruit_NeoPixel`	останній
WebSockets	`Links2004/arduinoWebSockets`	останній
ld2410	`ncmreynolds/ld2410`	останній (0.2.2 на момент написання)

Партиція

`min_spiffs` — Flash використано ~81%, ще є запас.

4️⃣ Веб-інтерфейс ESP32 (`192.168.0.40` чи актуальна IP)

Сторінка	Призначення
`/`	Головний дашборд — кнопки напоїв, живий статус (WebSocket, ~500мс)
`/info`	Система: QR-код, лічильник напоїв **(не скидається опівночі — немає RTC, лише при перезавантаженні!)**, час роботи, пам’ять, Wi-Fi/BLE сила сигналу, версія прошивки, `func/exec` наживо, посилання на детальний лог, рядок радара (клікабельний, кольоровий), повзунок яскравості RGB-індикатора
`/debug_log`	Кільцевий буфер останніх ~300 діагностичних рядків (пульс `loop()`, зміни `func/exec`, публікації `brew_progress`, збірка BLE-пакетів) — працює без USB, через веб
`/presence`	Радар: живі цифри (дистанція+сила окремо для руху/нерухомості, кольорові так/ні, іконка стану у верхньому куті), форма 9 налаштувань (мін/макс/час-спрацювання/затримка — окремо для “Рух” і “Присутність”, і спільна чутливість)

5️⃣ RGB-індикатор на самому ESP32

Один спільний 4-секундний цикл, без жодного `delay()` (усе — порівняння часу в `loop()`):

База (постійний колір): - Зелений — машина й телефон обидва підключені - Синій — лише машина підключена - Червоний, що блимає (раз на 500мс) — нічого не підключено

Поверх бази, у межах кожного 4-секундного циклу: | Момент циклу | Подія | |—|—| | 0-60мс | Білий спалах — Wi-Fi OK (жовтий замість білого, якщо Wi-Fi відвалився) | | 1000-1060мс, 1200-1260мс | Два бірюзових спалахи (200мс проміжок) — **присутність** (радар) | | 2000-2060мс, 2200-2260мс | Додатково два червоних спалахи (200мс проміжок) — **рух** (радар; показується разом із бірюзовими вище) |

Тип “присутність/рух” визначається один раз на початку кожного циклу (на позначці 1000мс), щоб не “стрибати” посеред самого патерну.

6️⃣ WLED-матриця 16×16 (окремий пристрій, керується через Home Assistant)

Коли машина активна (прогрівається/готує/щойно закінчила):

Preset	Що показує
1	Прогрів (анімація “Black Hole”)
2-12	Чашка кави, заповнення 0-100% (по 10%)
13	“Готовність”
14	Велика анімована крапля води (немає води)
15	Велика анімована іконка смітника (гуща повна)
16	Маленька іконка кавового зерна з написом “КАВА” (немає зерен)

Коли машина спить — показує погоду/годинник/дату циклічно (щохвилини перемикається: час →

дата → температура → вологість → тиск), з автоматичною яскравістю від датчика освітлення. Вимикається повністю після 3 годин темряви (якщо кухня вимкнена), знову вмикається на світанку чи коли кухню вмикають вручну вночі.

Захист від “фантомної зеленої чашки”

Консолідована автоматизація “Кавові анімації на матриці” тримає таймстемп останньої реальної події `brew_progress` (`input_number.ostannii_brew_progress_ts`). “Готовність” малюється (і зі стану, і з “серцебиття” сенсора попереджень) **лише** якщо з останнього реального `brew_progress` минуло 4+ хвилини — інакше шумові/фантомні переходи не перебивають анімацію наповнення. Виняток: перехід **із** “waking” (звичайне увімкнення без варіння) завжди малює “Готовність” миттєво, без затримки.

7️⃣ Ключові технічні знахідки й виправлені баги (хронологічно)

Ці пункти варто пам’ятати, якщо колись доведеться повертатись до старої версії чи діагностувати щось подібне знову:

1. **Self-poll під час варіння** — спочатку заборонили (щоб не заважати “живій трансляції” машини), потім з’ясували живими тестами: машина **не завжди** сама транслює під час варіння (підтверджено на гарячій воді — 14+ секунд повної тиші). Self-poll повернули активним і під час варіння, підстрахувавши лише перевіркою `fragCount==0`.
2. **`brewingActive` не мав шляху скинутись** — фундаментальний баг, що сидів із самого початку: прапорець скидався лише через реальну зміну рядка стану, а варіння завжди стартує вже зі стану “ready” (тобто “зміни” й не було). Виправлено прямим стеженням за сигналом “видача зупинилась” (`func==0` безперервно 15 секунд — час, а не кількість опитувань, бо короткі паузи всередині рецепта, наприклад преінфузія, могли хибно зарахуватись як “закінчилось”).
3. **Фрагментація BLE-пакетів під час високого навантаження** — межа “кінець пакета” визначалась 120мс тиші; під щільним трафіком варіння один реальний пакет міг розрізатись навпіл. Виправлено — межа тепер визначається за заявленою довжиною (`byte[1]`), а не за тишею.
4. **NOTIFY vs INDICATE** — спроба скопіювати офіційний CoffeeLink (підписка через INDICATE замість NOTIFY) **погіршила** якість прийому (більше побитих пакетів), тому відкочено назад на NOTIFY.
5. **USB CDC без підключеного хосту блокує `loop()` назавжди** — вимкнено (`CDCOnBoot=default`), додано `/debug_log` як заміну живій USB-діагностиці.
6. **`attention_type` не публікувався, якщо ніхто не тримав відкритою веб-сторінку** — вся логіка сиділа всередині функції, яка виходила одразу, якщо `webSocket.connectedClients()==0`. Винесено в окрему, незалежну функцію.
7. **Чутливість радара — контрінтуїтивна шкала:** менше число = чутливіший (спрацьовує на слабший сигнал), більше число = менш чутливий. Це не “сила”, а поріг (threshold), з яким порівнюється сила сигналу.

8️⃣ Радар присутності LD2410C — повний опис

Два незалежні виходи

- **“Рух”** (`radarMoveConfirmed`) — лише активний рух у заданому діапазоні дистанції.
- **“Присутність”** (`radarPresenceConfirmed`) — рух **АБО** нерухомий об’єкт, будь-який у своєму діапазоні.

Дебаунс (в обидва боки, окремо для кожного з двох виходів)

- **Час спрацювання** (вхід) — скільки секунд об’єкт має бути **безперервно видимим**, перш ніж

повірити, що це реально рух/присутність.
- **Затримка** (вихід) — скільки секунд тримати стан “так” після того, як об’єкт **реально зник**, перш ніж повірити, що його більше немає.
- Обидва параметри в `/presence` вводяться в **секундах** (з десятими, наприклад `0.5`).

Чутливість

Одне спільне число (0-100) на всі 9 зон одразу, записується напряму в сам сенсор (справжня команда, не програмний фільтр). **Менше число = чутливіше.**

MQTT

- `delonghi/radar_motion` (on/off, retained)
- `delonghi/radar_presence` (on/off, retained)
- Обидва мають повноцінний **MQTT discovery** (`binary_sensor`, `device_class: motion`/`occupancy`) — HA сама створює сутності.
- Значення публікуються одразу при підключенні до MQTT, а не лише на першу реальну зміну.

Живі дані без гальм

Радар опитується окремим, незалежним UART, не через BLE — не додає жодного навантаження на кавовий MITM-канал. `/presence` оновлюється кожні 500мс.

Важливо

У HA сенсор може одночасно існувати **у двох виглядах**: наш (через MQTT, стабільний, з дебаунсом) і, якщо випадково додано, штатна BLE-інтеграція HA (сирі, недебаунсовані дані — “моргає”). Використовуй наш, MQTT-based.

9️⃣ MQTT-топіки (повний список)

Топік	Опис
`delonghi/state`	asleep/waking/ready/offline (retained)
`delonghi/availability`	online/offline (LWT)
`delonghi/attention_type`	water/grounds/beans/“” (retained, серцебиття 30с)
`delonghi/brew_progress`	0-100, публікується на кожну реальну зміну
`delonghi/param_source`	“ESP32”/“Профіль N” (retained)
`delonghi/cmd`	команди: coffee/espresso/cappuccino/latte_macchiato/hot_water/*/stop/profile_cycle/power_on
`delonghi/radar_motion`	on/off (retained, з discovery)
`delonghi/radar_presence`	on/off (retained, з discovery)

🔟 Home Assistant — автоматизації

1. **“Кавові анімації на матриці”** (mode: restart) — уся логіка матриці, з таймстемп-гейтом проти фантомної зеленої чашки (розділ 6).
2. **“Погода — повне керування”** (mode: queued) — головна погодна логіка WLED, тригер раз на хвилину.
3. **“Голосові оголошення PrimaDonna”** (mode: queued) — “Прімадонна включилася”/“спить”/“готує каву”, з дедуплікацією проти хибних повторів при перезавантаженні HA (`context.user_id`/`from_state` перевірка).
4. **“Привітання та погода на кухні”** (mode: single) — вітання за часом доби + повна погода

словами (температура/вологість/тиск/вітер із напрямком і швидкістю з десятими, УФ-індекс, освітленість) — усі числа вимовляються **словами**, не цифрами (через Jinja-макрос, щоб TTS не плутав відмінки).

5. **“Освітлення кухні за присутністю”** (mode: queued) — вмикає світло кухні за радаром + порогом люксів (`input_number.porig_osvitlennia_kukhni`, повзунок 50-200), з потрійною перевіркою спрацювання (3 спроби, 3с пауза) і другим раундом перевірки через 2 хвилини для вимкнення. Поважає ручне втручання: якщо хтось вимкнув світло вручну, поки є присутність — автоматика не втручається аж до зникнення й повторної появи присутності (`input_boolean` прапорець).
6. **Кнопки-ярлики** — “Сказати погоду”/“Сказати стан машини”/“Сказати час” (Template Button, викликають відповідні автоматизації напряму, `skip_condition: true`).

Ключові хелпери

- `input_number.ostannii_brew_progress_ts` — таймстемп останнього реального `brew_progress`
- `input_number.porig_osvitlennia_kukhni` — поріг освітленості для світла кухні (50-200 люкс)
- `input_boolean.ruchne_vimknennia_svitla_kukhni` — прапорець “хтось вимкнув вручну”

🎀 Повна історія версій (v4.15 → v4.58)

Скорочено — детальний опис кожної версії в попередніх розмовах з Claude, ключові віхи:

- **v4.15** — базова робоча версія (варіння, веб, MQTT, OTA)
- **v4.16-v4.30** — self-poll фікси, детекція варіння (перші спроби), фільтр фантомного “asleep”, профілі по колу
- **v4.31-v4.39** — діагностика `func/exec`, `func==0` як сигнал сну, вузький захист від фантомного сну
- **v4.40-v4.43** — веб-незалежна публікація boot-стану, `/debug_log`, “пульс” `loop()`
- **v4.44-v4.48** — NOTIFY/INDICATE дослідження (відкочено на NOTIFY), межа пакета за довжиною, `brewingActive` тайм-аут фікс
- **v4.49** — `attention_type` винесено з-під залежності від WebSocket-клієнтів
- **v4.50** — час-базоване (не лічильник читань) визначення завершення варіння
- **v4.51-v4.52** — інтеграція радара LD2410C, фікс хибного “підключено”
- **v4.53-v4.56** — кольори на `/presence`/`/info`, MQTT discovery для радара, порядок елементів `/info`, секунди замість мс
- **v4.57-v4.58** — небклокуючі RGB-патерни присутності/руху на індикаторі, фінальні паузи між спалахами

Відомі відкриті питання

- `drinksToday` не скидається опівночі (немає RTC/NTP у прошивці) — лише при перезавантаженні. Заплановано або додати NTP+скидання, або перейменувати напис на “N напоїв до перезавантаження”.
- `func==7` для родини кави/еспресо підтверджено лише декомпіляцією офіційного застосунку, не власним живим тестом (на відміну від `func==11` для гарячої води, яке підтверджено напряму).